Subject: Programming with Java
ASSIGNMENT No : 1 **TITLE :** Develop an application based on Java environment setup, JDK/JRE/JVM, variables, methods and constructors. and demonstrate the implementation of the concepts through suitable examples.

Problem Statement: Set up a working Java development environment, understand how JDK, JRE, and JVM relate to one another, and build a small program that puts variables, methods, and constructors to use.

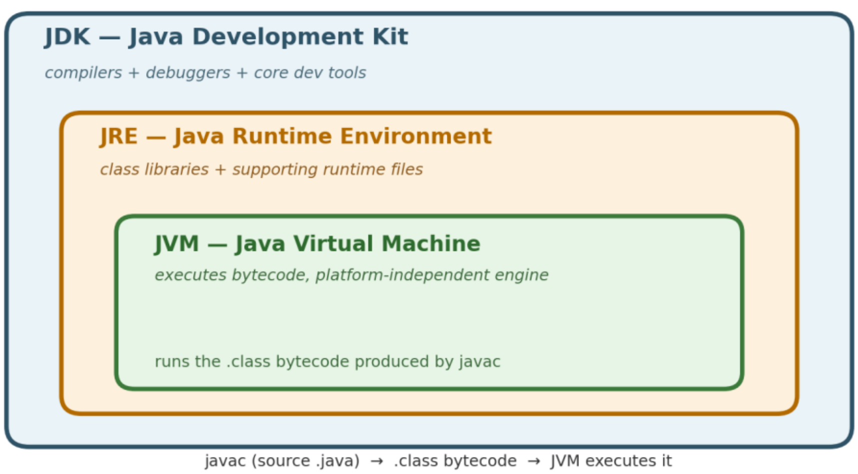
Learning Objectives

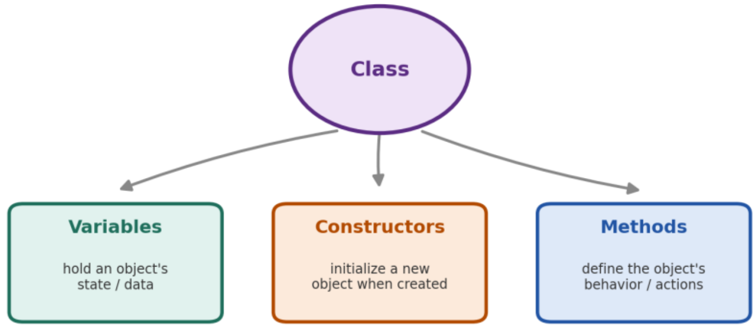
Understand the theoretical concepts, implement them in Java programs, analyze outputs, and apply the concepts to solve practical programming problems.

Theory

Java programming begins with properly setting up the development environment, along with building a solid understanding of core concepts such as JDK, JRE, JVM, variables, methods, and constructors. Students should aim to understand not just how these concepts work, but why they work that way — including the underlying architecture, their advantages, common use cases, and how they're applied in practice. This involves grasping the design principles behind the language, following good coding practices, understanding the program's execution flow, and recognizing how these concepts are used in real-world software development. A well-rounded study should therefore cover the syntax, relevant APIs, execution process, and practical applications of these fundamentals.

How pieces of Java relate to each other:

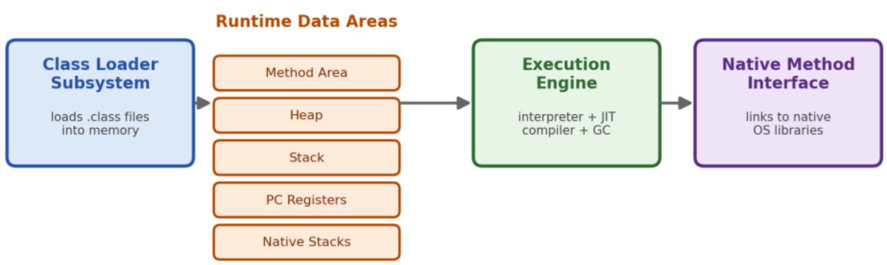




Together these building blocks define how an object stores data and behaves within a Java program.

Here is how the JVM is structured internally:

JVM Architecture (execution flow)



Bytecode moves left to right: loaded, staged in runtime memory, executed, then bridged to native system resources when needed.

Code:

Product.java

```java
public class Product {
    String productName;
    long productCode;
    double price;

    // Constructor: sets initial state of a new object
    Product(String productName, long productCode, double price) {
        this.productName = productName;
        this.productCode = productCode;
        this.price = price;
    }

    // Method: defines behavior of the object
    void showDetails() {
        System.out.println("Product Name: " + productName);
        System.out.println("Product Code: " + productCode);
        System.out.println("Price: Rs. " + price);
    }

    public static void main(String[] args) {
        Product p1 = new Product("Mechanical Keyboard", 987654321012L, 4999.0);
        p1.showDetails();
    }
}
```

Output Screenshot

Terminal – Product

```
$ java Product
Product Name: Mechanical Keyboard
Product Code: 987654321012
Price: Rs. 4999.0
```

Conclusion

The Java development environment was configured successfully — the VS Code settings.json file was updated to reference a proper JDK 17+ installation, which resolved a language-server crash that had been occurring in the background. A full sample application was then written and executed to demonstrate how classes, constructors, methods, and variables interact in practice. During this process, a numeric overflow bug was also fixed by changing the product code field from a 32-bit int to a 64-bit long, allowing it to safely store the complete value. Two additional issues were resolved along the way — a background process that had entered a deadlock, and a typo in a flag used during version control setup — with the project ultimately being pushed cleanly to GitHub.

Questions

1. Explain the main concept used in Java environment setup, variables and methods.

Ans.

- **Environment Setup:** To build Java applications, source code written in human-readable form needs to be compiled into bytecode using the JDK (Java Development Kit). The JRE (Java Runtime Environment), in turn, packages the essential class libraries along with the JVM (Java Virtual Machine), which is responsible for executing that bytecode — this is what makes Java software platform-independent.
- **Variables:** These act as named storage locations in system memory, used to hold values of a particular data type (like String or long), and are essentially what keeps track of an object's state at any given point during execution.

- **Methods:** These are self-contained blocks of code built to carry out specific tasks or logical operations. They define how a class behaves and help keep the overall code organized and reusable.

2. What are the real-world applications?

Ans.

- **Banking & E-Commerce Backends**: Java is a go-to choice for building secure, scalable backend systems used in large-scale platforms like banking, online retail, and booking/reservation systems (e.g., managing user profiles and processing data).
- **Mobile App Development:** Native Android applications are built heavily around Java's class-based architecture, using constructors to initialize objects and methods to handle UI interactions and behaviour.
- **Distributed Computing & Cloud Platforms:** Widely used big data frameworks like Apache Hadoop and Apache Spark rely on Java to power intensive computation pipelines and support high-throughput, large-scale server infrastructure.